

# SECURITE HARDWARE

RATTRAPAGE OPTION CYBERSECURITE

Etudiant :  
Dede Nerista Aline TETE BENISSAN

Professeur :  
Julien FRANCO

# SOMMAIRE

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>I. Introduction.....</b>                                                | <b>2</b>  |
| Figure 1 : Arborescence du projet.....                                     | 3         |
| <b>II. Rappels sur AES-128.....</b>                                        | <b>3</b>  |
| Figure 2 : Données initiales et clés de ronde AES-128.....                 | 4         |
| <b>III. Principe des attaques en faute.....</b>                            | <b>4</b>  |
| Figure 3 : Injection de faute avant MixColumns de la ronde 9.....          | 5         |
| <b>IV. Modèle de faute utilisé.....</b>                                    | <b>5</b>  |
| Figure 4 : Implémentation du modèle de faute dans AES.....                 | 6         |
| <b>V. Attaque 1 : Faute avant MixColumns de la ronde 9.....</b>            | <b>6</b>  |
| Figure 5.1 : Filtrage des candidats de K10 dans l'attaque 1.....           | 6         |
| Figure 5.2 : Résultat de l'attaque 1 et récupération de K0.....            | 7         |
| <b>VI. Attaque 2 : faute avant MixColumns de la ronde 8.....</b>           | <b>7</b>  |
| Figure 6.1 : une faute ronde 8 impacte les 16 octets après la ronde 9..... | 7         |
| Figure 6.2 : Resultat.....                                                 | 8         |
| <b>VII. Récupération de K10.....</b>                                       | <b>8</b>  |
| <b>VIII. Récupération de K0 avec InvKeySchedule.....</b>                   | <b>8</b>  |
| <b>IX. Implémentation Python.....</b>                                      | <b>9</b>  |
| <b>X. Résultats expérimentaux.....</b>                                     | <b>9</b>  |
| <b>XI. Comparaison des deux attaques.....</b>                              | <b>10</b> |
| Figure 7 : Resultat comparatif.....                                        | 10        |
| <b>XII. Conclusion.....</b>                                                | <b>10</b> |

# I. Introduction

AES est un algorithme de chiffrement symétrique très utilisé pour protéger des données. Il chiffre des blocs de 128 bits à l'aide d'une clé secrète. Dans le cas d'AES-128, la clé fait 128 bits et le chiffrement comporte 10 rondes.

Les attaques classiques contre AES cherchent généralement à retrouver la clé par analyse mathématique ou par recherche exhaustive. Les attaques en faute utilisent une autre approche : elles consistent à perturber volontairement l'exécution du chiffrement afin d'observer les effets de cette perturbation sur le texte chiffré final.

Dans ce projet, on s'intéresse à l'attaque différentielle en faute de Piret/Quisquater, notée PQ03. Notre objectif est d'injecter une faute dans un octet de l'état interne de l'AES, avant l'opération MixColumns, puis d'exploiter les différences entre un texte chiffré correct et des textes chiffrés fautés afin de retrouver la clé de dernière ronde K10.

Une fois K10 retrouvée, on utilise l'inverse du KeySchedule pour retrouver la clé secrète initiale K0.

Deux variantes sont étudiées :

**Attaque 1** : faute aléatoire dans un octet avant le MixColumns de la ronde 9.

**Attaque 2** : faute aléatoire dans un octet avant le MixColumns de la ronde 8.

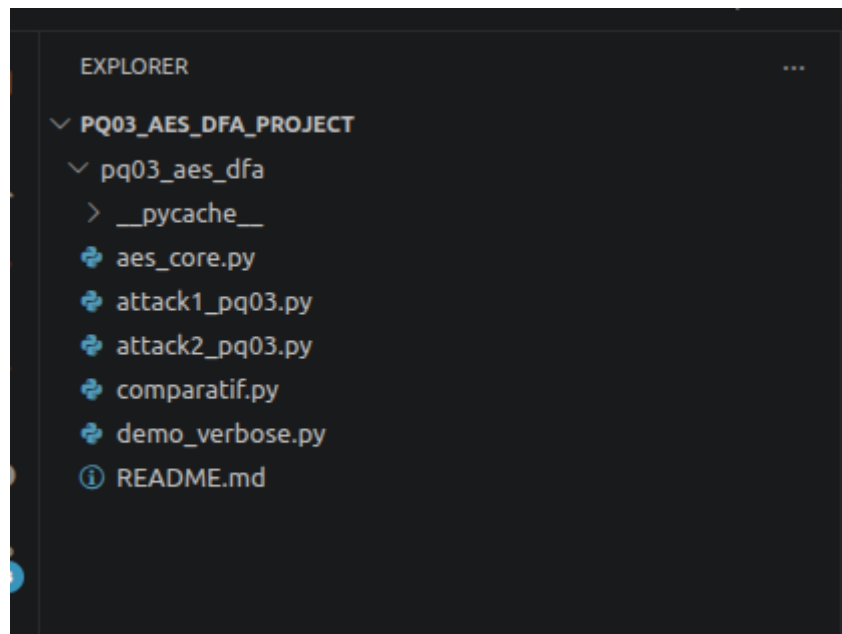


Figure 1 : Arborescence du projet

## II. Rappels sur AES-128

AES-128 chiffre des blocs de 128 bits, c'est-à-dire 16 octets. Ces 16 octets sont représentés sous forme d'une matrice de taille 4x4 appelée état AES.

Chaque ronde AES applique plusieurs transformations :

**SubBytes** : chaque octet est remplacé par une valeur issue de la SBOX.

**ShiftRows** : les lignes de la matrice sont décalées.

**MixColumns** : les octets de chaque colonne sont mélangés pour assurer la diffusion.

**AddRoundKey** : l'état est combiné avec une clé de ronde par XOR.

AES-128 utilise une clé initiale K0, puis génère les clés de ronde K1 à K10 grâce au KeySchedule. La dernière ronde est particulière, car elle ne contient pas l'opération MixColumns.

```
● qlivai@qlivai:~/securite hardware/pq03_aes_dfa_project/pq03_aes_dfa$ python3 demo_verbose.py | head -80
=== Données initiales ===
Texte clair      : 00112233445566778899aabbccddeeff
Clé secrète K0   : 2b7e151628aed2a6abf7158809cf4f3c

Clés de ronde AES-128
K00              : 2b7e151628aed2a6abf7158809cf4f3c
                  [2b 28 ab 09]
                  [7e ae f7 cf]
                  [15 d2 15 4f]
                  [16 a6 88 3c]
K01              : a0fafe1788542cb123a339392a6c7605
                  [a0 88 23 2a]
                  [fa 54 a3 6c]
                  [fe 2c 39 76]
                  [17 b1 39 05]
K02              : f2c295f27a96b9435935807a7359f67f
                  [f2 7a 59 73]
                  [c2 96 35 59]
                  [95 b9 80 f6]
                  [f2 43 7a 7f]
K03              : 3d80477d4716fe3e1e237e446d7a883b
                  [3d 47 1e 6d]
                  [80 16 23 7a]
                  [47 fe 7e 88]
                  [7d 3e 44 3b]
K04              : ef44a541a8525b7fb671253bdb0bad00
                  [ef a8 b6 db]
                  [44 52 71 0b]
                  [a5 5b 25 ad]
                  [41 7f 3b 00]
K05              : d4d1c6f87c839d87caf2b8bc11f915bc
                  [d4 7c ca 11]
                  [d1 83 f2 f9]
```

```
qlivai@qlivai:~/securite hardware/pq03_aes_dfa_project/pq03_aes_dfa$ python3 demo verbose.py | head -80
[d1 83 f2 f9]
[c6 9d b8 15]
[f8 87 bc bc]
K06      : 6d88a37a110b3efddb98641ca0093fd
          [6d 11 db ca]
          [88 0b f9 00]
          [a3 3e 86 93]
          [7a fd 41 fd]
K07      : 4e54f70e5f5fc9f384a64fb24ea6dc4f
          [4e 5f 84 4e]
          [54 5f a6 a6]
          [f7 c9 4f dc]
          [0e f3 b2 4f]
K08      : ead27321b58dbad2312bf5607f8d292f
          [ea b5 31 7f]
          [d2 8d 2b 8d]
          [73 ba f5 29]
          [21 d2 60 2f]
K09      : ac7766f319fadc2128d12941575c006e
          [ac 19 28 57]
          [77 fa d1 5c]
          [66 dc 29 00]
          [f3 21 41 6e]
K10      : d014f9a8c9ee2589e13f0cc8b6630ca6
          [d0 c9 e1 b6]
          [14 ee 3f 63]
          [f9 25 0c 0c]
          [a8 89 c8 a6]
Texte chiffré correct : 8df4e9aac5c7573a27d8d055d6e4d64b
```

*Figure 2 : Données initiales et clés de ronde AES-128*

### III. Principe des attaques en faute

Une attaque en faute consiste à modifier volontairement une valeur interne pendant le chiffrement. L'attaquant compare ensuite deux résultats :

le texte chiffré correct C,

le texte chiffré fauté C'.

La différence entre C et C' donne des informations sur l'état interne de l'AES. Ces informations permettent ensuite de filtrer des hypothèses de clé.

Dans ce projet, la faute est modélisée par un XOR sur un octet de l'état AES. La position de l'octet fauté et la valeur de la faute sont aléatoires.

La faute est injectée avant l'opération MixColumns, soit à la ronde 9, soit à la ronde 8.

```

Injection de faute
Ronde ciblée      : 9
Position octet     : 12
Valeur originale   : 7a
Masque XOR faute   : a0
Valeur fautée     : da
État avant faute   : 07e12a3d542a0a44539a837f7a3cc1b6
                   [07 54 53 7a]
                   [e1 2a 9a 3c]
                   [2a 0a 83 c1]
                   [3d 44 7f b6]
État après faute   : 07e12a3d542a0a44539a837fda3cc1b6
                   [07 54 53 da]
                   [e1 2a 9a 3c]
                   [2a 0a 83 c1]
                   [3d 44 7f b6]

```

*Figure 3 : Injection de faute avant MixColumns de la ronde 9*

## IV. Modèle de faute utilisé

Le modèle de faute utilisé est le suivant :

- Faute aléatoire sur un seul octet
- Injection avant MixColumns
- Masque XOR non nul
- Position aléatoire dans l'état AES

Dans le code, la faute est représentée par la classe **Fault**.

Celle ci contient :

- le numéro de ronde ciblée,
- la position de l'octet fauté,
- la valeur XOR de la faute,
- l'indication que la faute est injectée avant MixColumns.

La faute est donc de la forme :

**état[position] = état[position] XOR faute**

Ce modèle correspond au scénario demandé pour l'attaque de Piret/Quisquater.

```

177         print_state('Après ShiftRows', s)
178         if fault and fault.round_no==rnd and fault.before_mixcolumns:
179             # Injection DFA : on modifie un octet de l'état par XOR.
180             # C'est un modèle classique de faute aléatoire sur un octet.
181             pos = random.randrange(16) if fault.byte_pos is None else fault.byte_pos
182             val = random.randrange(1,256) if fault.value is None else fault.value
183             before_fault=s[:]
184             original=s[pos]
185             s[pos]^=val

```

Figure 4 : Implémentation du modèle de faute dans AES

## V. Attaque 1 : Faute avant MixColumns de la ronde 9

Dans la première attaque, la faute est injectée avant le MixColumns de la ronde 9.

À ce moment du chiffrement, une faute sur un octet se propage dans une seule colonne après MixColumns. Après la dernière ronde AES, cette faute apparaît sur 4 octets du texte chiffré final.

Ainsi, chaque texte chiffré fauté donne de l'information sur un groupe de 4 octets de la clé de dernière ronde K10.

Le programme compare le texte chiffré correct et le texte chiffré fauté. Il identifie les octets impactés, puis teste des hypothèses sur les 4 octets correspondants de K10.

Pour chaque hypothèse de clé, le programme inverse partiellement la dernière ronde AES :

$\text{InvSbox}(C\_i \text{ XOR } k\_i) \text{ XOR } \text{InvSbox}(C'\_i \text{ XOR } k\_i)$

Si la différence obtenue correspond à un motif possible après MixColumns, le candidat est conservé. Sinon, il est éliminé.

Plusieurs fautes sont nécessaires, car une seule faute ne suffit généralement pas à obtenir un candidat unique. Les ensembles de candidats se croisent donc après chaque faute.

```
Texte chiffré fauté généré : def4e9aac5c7579f27d8f355d684d64b
Octets impactés           : [0, 7, 10, 13]
Colonne 0 positions [0, 13, 10, 7]
  Taille des candidats pour cette faute : 960 [00bc5a5b, 00bc5afe, 00bc795b, 00bc79fe, 00dc5a5b, 00dc5afe, ...]
  Réduction des candidats                : 992 -> 1
Colonne 1: aucune différence, colonne ignorée
Colonne 2: aucune différence, colonne ignorée
Colonne 3: aucune différence, colonne ignorée
[A1] faute 16: tailles des ensembles de candidats = [1, 1, 1, 1]
```

Figure 5.1 : Filtrage des candidats de K10 dans l'attaque 1

```
Résultat Attaque 1
K10 retrouvé : d014f9a8c9ee2589e13f0cc8b6630ca6
K10 réel      : d014f9a8c9ee2589e13f0cc8b6630ca6
K0 retrouvé  : 2b7e151628aed2a6abf7158809cf4f3c
K10 retrouvé : d014f9a8c9ee2589e13f0cc8b6630ca6
K10 réel      : d014f9a8c9ee2589e13f0cc8b6630ca6
K0 retrouvé  : 2b7e151628aed2a6abf7158809cf4f3c
K0 réel       : 2b7e151628aed2a6abf7158809cf4f3c
Textes fautés utilisés : 16
```

Figure 5.2 : Résultat de l'attaque 1 et récupération de K0

## VI. Attaque 2 : faute avant MixColumns de la ronde 8

Dans la deuxième attaque, la faute est injectée une ronde plus tôt, avant l'opération MixColumns de la ronde 8.

Contrairement à l'attaque précédente, la faute traverse un plus grand nombre de transformations AES. Après son injection, elle se propage à travers les opérations de la ronde 8 puis de la ronde 9 avant d'atteindre la dernière ronde.

Cette propagation entraîne une diffusion importante de l'erreur dans le texte chiffré final. Les différences observées entre le texte chiffré correct et les textes chiffrés fautés fournissent ainsi davantage d'informations exploitables pour la récupération de la clé de dernière ronde K10.

Chaque texte fauté permet de réduire progressivement l'ensemble des candidats possibles pour les différents groupes d'octets de K10. La répétition de cette opération sur plusieurs fautes conduit finalement à l'identification d'une unique clé compatible avec l'ensemble des observations.

Cette approche nécessite généralement moins de textes fautés que l'attaque de la ronde 9, mais elle implique un traitement plus complexe en raison de la diffusion plus importante de la faute au sein du chiffrement AES.

```
Octets impactés      : [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
Propagation          : une faute ronde 8 impacte les 16 octets après la ronde 9
Colonne 0 positions [0, 13, 10, 7]
  Taille des candidats pour cette faute : 992 [0149c3e6, 0149c3fa, 0149dce6, 0149dcfa, 016cc3e6, 016cc3fa, ...]
  Filtrage progressif : None -> 992
Colonne 1 positions [4, 1, 14, 11]
  Taille des candidats pour cette faute : 992 [00201530, 00201534, 00208d30, 00208d34, 00ca1530, 00ca1534, ...]
  Filtrage progressif : None -> 992
Colonne 2 positions [8, 5, 2, 15]
  Taille des candidats pour cette faute : 1040 [011e7d64, 011e7da4, 011ec264, 011ec2a4, 01c67d64, 01c67da4, ...]
  Filtrage progressif : None -> 1040
```

*Figure 6.1 : une faute ronde 8 impacte les 16 octets après la ronde 9*

```
Résultat Attaque 2
K10 retrouvé : d014f9a8c9ee2589e13f0cc8b6630ca6
K10 réel      : d014f9a8c9ee2589e13f0cc8b6630ca6
K0 retrouvé  : 2b7e151628aed2a6abf7158809cf4f3c
K10 retrouvé : d014f9a8c9ee2589e13f0cc8b6630ca6
K10 réel      : d014f9a8c9ee2589e13f0cc8b6630ca6
K0 retrouvé  : 2b7e151628aed2a6abf7158809cf4f3c
K0 réel      : 2b7e151628aed2a6abf7158809cf4f3c
Textes fautés utilisés : 2
```

*Figure 6.2 : Resultat*



## VII. Récupération de K10

Les deux attaques cherchent d'abord à retrouver la dernière clé de ronde K10.

Pour cela, le programme travaille sur des groupes de 4 octets. Il teste des hypothèses de clé et vérifie si la différence obtenue est compatible avec les motifs attendus par l'attaque PQ03.

Les positions utilisées sont les groupes suivants :

```
COL_CT_POS = [  
    [0,13,10,7],  
    [4,1,14,11],  
    [8,5,2,15],  
    [12,9,6,3]  
]
```

Chaque groupe correspond à une colonne de l'état AES avant le dernier ShiftRows.

Quand chaque groupe est réduit à un seul candidat, les 16 octets de K10 sont retrouvés.

## VIII. Récupération de K0 avec InvKeySchedule

L'attaque ne retrouve pas directement la clé secrète initiale K0. Elle retrouve d'abord K10, c'est-à-dire la clé de dernière ronde.

Cependant, le KeySchedule AES-128 est inversible. Il est donc possible de partir de K10 et de remonter jusqu'à K0.

Le programme implémente cette étape avec la fonction :

```
inv_key_schedule(k10)
```

Une fois K10 retrouvée, cette fonction permet de retrouver la clé initiale K0.

Dans les résultats, on vérifie que :

K10 retrouvé = K10 réel

K0 retrouvé = K0 réel

## IX. Implémentation Python

Le projet est organisé en plusieurs fichiers :

- **aes\_core.py**

Ce fichier contient l'implémentation AES-128, le KeySchedule, l'InvKeySchedule et l'injection de faute.

- **attack1\_pq03.py**

Ce fichier implémente l'attaque 1 avec une faute avant MixColumns de la ronde 9.

- **attack2\_pq03.py**

Ce fichier implémente l'attaque 2 avec une faute avant MixColumns de la ronde 8.

- **comparatif.py**

Ce fichier compare les deux attaques en affichant le nombre de textes fautés, la complexité approximative et le temps d'exécution.

- **demo\_verbose.py**

Ce fichier lance une démonstration complète avec affichage détaillé des états AES, des fautes, des candidats et des résultats.

## X. Résultats expérimentaux

Les expérimentations réalisées montrent que les deux variantes de l'attaque permettent de reconstruire correctement la clé de dernière ronde K10. Une fois cette clé obtenue, l'application de l'algorithme inverse du KeySchedule permet de retrouver la clé maîtresse K0 utilisée pour le chiffrement.

Exemple de résultat obtenu pour l'attaque 1 :

K10 obtenue : d014f9a8c9ee2589e13f0cc8b6630ca6

K10 réelle : d014f9a8c9ee2589e13f0cc8b6630ca6

K0 obtenue : 2b7e151628aed2a6abf7158809cf4f3c

K0 réelle : 2b7e151628aed2a6abf7158809cf4f3c

Exemple de résultat obtenu pour l'attaque 2 :

Textes fautés utilisés : 2

Octets impactés min/moy/max : 16 / 16 / 16

K10 obtenue : d014f9a8c9ee2589e13f0cc8b6630ca6

K10 réelle : d014f9a8c9ee2589e13f0cc8b6630ca6

K0 obtenue avec InvKeySchedule : 2b7e151628aed2a6abf7158809cf4f3c

Les résultats peuvent légèrement varier d'une exécution à l'autre puisque les fautes injectées sont générées aléatoirement. Cependant, les différentes expérimentations conduisent systématiquement à la reconstruction correcte des clés lorsque suffisamment de textes fautés sont disponibles.

## XI. Comparaison des deux attaques

L'attaque 1 injecte la faute plus tard, à la ronde 9. La diffusion est donc limitée : seulement 4 octets du texte chiffré sont impactés. Chaque faute filtre une seule colonne de K10.

L'attaque 2 injecte la faute plus tôt, à la ronde 8. La faute se diffuse sur les 16 octets du texte chiffré. Chaque faute permet donc de filtrer les 4 colonnes de K10.

On remarque que l'attaque 2 utilise souvent moins de textes fautés, car chaque faute donne plus d'information. Cependant, son coût par faute est plus élevé.

```

=== Tableau comparatif final ===
| Attaque | Ronde ciblée | Textes fautés | Colonnes/faute | Tests approx./faute | Octets impactés moy. | Temps (s) | Clé retrouvée |
|---|---|---|---|---|---|---|---|
| Attaque 1 | 9 | 10 | 1 | 131072 | 4.00 | 1.947 | oui |
| Attaque 2 | 8 | 2 | 4 | 524288 | 16.00 | 1.409 | oui |

```

*Figure 7 : Resultat comparatif*

## XII. Conclusion

Ce projet a permis d'étudier et d'implémenter deux variantes de l'attaque différentielle en faute de Piret et Quisquater appliquée à AES-128.

La première variante repose sur l'injection d'une faute avant l'opération MixColumns de la ronde 9. La propagation limitée de la faute permet de cibler progressivement les différentes colonnes de la clé de dernière ronde K10.

La seconde variante injecte la faute avant le MixColumns de la ronde 8. La diffusion plus importante de l'erreur dans le chiffrement fournit davantage d'informations exploitables, ce qui permet une réduction plus rapide de l'espace des candidats de clé.

Dans les deux cas, les expérimentations montrent qu'il est possible de reconstruire la clé de dernière ronde K10 puis de retrouver la clé maîtresse K0 grâce à l'algorithme inverse du KeySchedule.

Ces travaux illustrent qu'un algorithme cryptographique robuste comme AES peut devenir vulnérable lorsque son exécution physique est perturbée. Des contre-mesures telles que la détection de faute, la redondance de calcul ou la vérification des résultats sont donc indispensables pour renforcer la sécurité des implémentations matérielles.